P1610R0 Rename `await_resume()` to `await_result()`

By: Mathias Stearn (MongoDB)
Date: 2019-03-10

What?

The coroutine feature recently approved for C++20 defines an API for types that support being `co_await`ed, consisting of three methods: `await_ready()`, `await_suspend()`, and `await_resume()`. This paper proposes to rename the last to `await_result()`, which better reflects its purpose and effects.

Why?

- What does this method do? It returns the **result** of the co_**await** expression!
- `await_resume()` is called even when the coroutine isn't suspended if `await_ready()` returns `true`, so there is no resuming to be done.
- It looks like it has symmetry with `await_suspend()`, but that's a lie. They are almost completely unrelated.
- It is attached to Awaitables, but you don't resume Awaitables, you resume coroutines. This subject-object confusion is particularly problematic when writing a coroutine type that is also an Awaitable, such as `Future` or `Task`. When you define the `await_resume()` method for such a type, you need to remember that it isn't called when the coroutine that your are defining is resumed. Instead, it is called when the consuming coroutine resumes, to get the *result* of this coroutine.

Summary: This change makes the Awaitable API more intuitive, therefore easier to teach and learn. This is important because that API is the second layer in the "Gor Table" from section 4.4 of P1362, so it will be the first form of coroutine customization most users will do.

Why not?

- Incompatibility with existing code written against the TS. However,
  - We don't promise compatibility for TSs.
  - Implementations would be allowed (encouraged?) to fallback to the old name if the new one isn't present for some transition period.

What to do about the TS?

If we want to ease the transition, we can add explicit wording to the TS to work with both names. We would need to either prefer one if both are present, or just make that ill-formed.